

iStore LK Software Requirements Specification

Apple Products E-Commerce & Retail Management System
Angular · Spring Boot · MongoDB

Document Version	1.1
Date	July 14, 2026
Status	Draft – For Review
Technology Stack	Angular 17 · Spring Boot 3 · MongoDB 7
Prepared By	iStore LK Engineering Team

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions & Acronyms
 - 1.4 Document Conventions
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 User Classes & Characteristics
 - 2.3 Operating Environment
 - 2.4 Assumptions & Dependencies
3. System Features
 - 3.1 Customer Storefront
 - 3.2 Product & Variant Management
 - 3.3 Shopping Cart & Checkout
 - 3.4 Order Management
 - 3.5 Inventory & Stock Control
 - 3.6 Trade-In & Pre-Orders
 - 3.7 Admin Dashboard & Analytics
 - 3.8 Notifications & Alerts
4. Functional Requirements
5. Non-Functional Requirements
6. External Interface Requirements
7. Database Design Overview (MongoDB)
8. Technology Stack
9. Constraints & Risks
10. Appendix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **iStore LK**, an Apple-products-only e-commerce and retail management system. The document serves as the primary agreement between stakeholders and the development team regarding what the system must do. It covers the customer-facing storefront and the internal administrative panel used by store staff and management.

1.2 Scope

iStore LK is a full-stack web application built with **Angular 17** (frontend), **Spring Boot 3** (REST API backend), and **MongoDB 7** (document database). It manages the complete lifecycle of Apple product retail — from catalogue browsing with colour/storage variant selection, through cart, checkout and order fulfilment, to SKU-level inventory control, trade-in evaluation, warranty registration, and sales analytics. The system targets authorised Apple resellers operating in Sri Lanka.

Key capabilities in scope:

- Customer storefront: category browsing (iPhone, Mac, iPad, Watch, AirPods, Vision Pro, TV & Home, Accessories), variant selector (colour + storage/RAM), product comparison, reviews
- Shopping cart, multi-step checkout, island-wide delivery or store-pickup options
- Order lifecycle management with status tracking and PDF invoices
- SKU-level inventory per variant with low-stock alerts and stock movement audit trail
- Pre-order management for new Apple product launches
- Trade-in request and evaluation workflow (device condition grading, quote, credit)
- Warranty registration by serial number and claim/repair ticket tracking
- Role-based access control (CUSTOMER, STAFF, ADMIN)
- Email/SMS notification triggers for order and stock events
- Sales and revenue analytics dashboard with charts

1.3 Definitions & Acronyms

SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
API	Application Programming Interface (Spring Boot REST layer)
JWT	JSON Web Token — used for stateless authentication
LKR	Sri Lankan Rupee — primary currency unit
SKU	Stock Keeping Unit — a unique sellable variant (e.g., iPhone 16 Pro, 256 GB, Blue Titanium)
SPA	Single-Page Application (Angular)
BSON	Binary JSON — MongoDB's document storage format
ODM	Object-Document Mapping (Spring Data MongoDB)
CRUD	Create, Read, Update, Delete — standard data operations
IMEI	International Mobile Equipment Identity — device identifier for iPhones
PDPA	Personal Data Protection Act (Sri Lanka)

1.4 Document Conventions

- Requirements are identified as FR-XXX (functional) or NFR-XXX (non-functional).
- Priority levels: High (must-have), Medium (should-have), Low (nice-to-have).

- All monetary values are in LKR unless stated otherwise.
- Role names (CUSTOMER, STAFF, ADMIN) match the enum in the backend.
- “Product” refers to a model (e.g., iPhone 16 Pro); “variant” refers to a specific sellable configuration (colour + storage) identified by a SKU.

2. Overall Description

2.1 Product Perspective

iStore LK is a standalone, cloud-deployable web application. The Angular SPA communicates exclusively with the Spring Boot REST API over HTTPS. The API persists data to a MongoDB 7 replica set using Spring Data MongoDB. A separate email/SMS gateway (e.g., SendGrid / Dialog SMS) is integrated for notifications. Product images and trade-in device photos are stored on a cloud object store (e.g., AWS S3), with only URLs stored in the database.

2.2 User Classes & Characteristics

Customer — Registered end-user who browses the catalogue, selects variants, places orders, submits trade-in requests, registers warranties, and manages their profile. Primarily a mobile-friendly experience.

Staff — Store employee who creates/manages orders on behalf of walk-in customers, processes in-store pickups, verifies trade-in devices, and updates order statuses.

Admin — Store owner/administrator who manages the product catalogue, pricing, stock levels, pre-order campaigns, approves trade-in quotes, and has full access to reports, user management, system configuration, and audit logs.

2.3 Operating Environment

Frontend	Angular 17 SPA — Chrome, Firefox, Safari, Edge (latest 2 versions)
Backend	Spring Boot 3.x on JDK 21, packaged as Docker image
Database	MongoDB 7.0+ — 3-node replica set (or MongoDB Atlas M10+)
Hosting	Cloud VM or Kubernetes cluster; HTTPS enforced
Auth	JWT Bearer tokens, refresh-token rotation, 15-min access token expiry

2.4 Assumptions & Dependencies

- All products are genuine Apple devices sourced through an authorised distributor; the system stores serial numbers/IMEI at the point of sale.
- All users have access to a modern web browser and a stable internet connection.
- The company provides a valid SSL certificate for the production domain.
- Third-party payment gateway (PayHere) must provide a sandbox for testing (Phase 2).
- Product images are managed by the Admin role; the system stores image URLs, not binary files.
- SMS/email notification service credentials are provided by the business.
- LKR is the only supported currency in V1; multi-currency support is out of scope.
- Apple's official pricing and product data are entered manually; no integration with Apple APIs.

3. System Features

3.1 Customer Storefront

The public-facing Angular interface where customers discover and purchase Apple products.

- Hero section with featured product banners (new launches, promotions)
- Category navigation strip: All, iPhone, Mac, iPad, Watch, AirPods, Vision Pro, TV & Home, Accessories
- Product grid with image, starting price (“From LKR ...”), availability badge, and rating
- Product detail page: image gallery, Apple-style colour swatch and storage picker, dynamic price per selected variant, tech specs table, stock indicator, customer reviews
- Compare view: side-by-side spec comparison of up to 3 products in the same category
- Global search with autocomplete (product name, chip, model number)
- My Orders section: order list with status badges, tracking, and invoice download
- Customer profile management (name, NIC, contact info, delivery addresses)
- Wishlist with back-in-stock notification opt-in

3.2 Product & Variant Management

Admin tools for maintaining the Apple-only catalogue.

- Add / edit / archive products with: name, category, model number, chip, description, release year, base specs, images
- Variant matrix per product: colour × storage/RAM combinations, each with its own SKU, price, and stock quantity
- Category-specific spec templates (iPhone: chip/camera/battery; Mac: RAM/SSD/display; AirPods: ANC/battery hours) enabled by MongoDB's flexible schema
- Condition flag per variant: Brand New, Refurbished, Pre-Owned (graded A/B/C)
- Price history log per SKU for margin analysis
- Product status: Active, Coming Soon (pre-order), Out of Stock, Archived
- Bulk price update by category or product line

3.3 Shopping Cart & Checkout

- Persistent cart per customer (stored server-side, merged on login)
- Cart line items reference a specific SKU with quantity, unit price, and line total
- Multi-step checkout: cart review → delivery or store pickup → payment method → confirm
- Delivery fee calculated by district; free delivery above a configurable threshold (e.g., LKR 50,000)
- Payment methods: Cash on Delivery, Bank Transfer (slip upload), Card via PayHere (Phase 2), Koko / Mintpay instalment (Phase 2)
- Promo code support: percentage or fixed discount with validity window and usage limit
- Stock re-validation at checkout to prevent overselling

3.4 Order Management

- Order status lifecycle: Pending → Confirmed → Processing → Shipped / Ready for Pickup → Delivered / Collected → Completed; Cancelled at permitted stages
- Order ID format: #IS-YYYY-XXXXX (year + sequential number)
- Staff can create orders for walk-in customers (POS-style flow)
- Serial number / IMEI capture at fulfilment for warranty linkage
- PDF invoice generation for every confirmed order
- Customer cancellation allowed until the order reaches Processing status
- Return/refund request workflow with reason and approval step

3.5 Inventory & Stock Control

- Real-time stock quantity per SKU with reserved-stock handling during checkout
- Atomic stock decrement on order confirmation to prevent race conditions
- Stock movement ledger: purchase-in, sale, return, adjustment, trade-in intake
- Low-stock threshold per SKU with dashboard alert and email to Admin
- Goods-received entry: supplier, quantity, unit cost, batch reference
- Stock valuation report (quantity × cost) by category

3.6 Trade-In & Pre-Orders

- Customer submits trade-in request: device model, storage, condition questionnaire, photos
- Staff grades the device (A/B/C/Reject) and issues a quote in LKR
- Accepted trade-in value applied as credit toward a new purchase
- Trade-in intake adds the device to Pre-Owned inventory after refurbishment check
- Pre-order campaigns for upcoming launches: deposit amount, expected availability date, queue position, automatic conversion to order on stock arrival

3.7 Admin Dashboard & Analytics

- KPI cards: Total Revenue (LKR), Orders Today, Low-Stock SKUs, New Customers
- Bar chart: monthly revenue trend (current year)
- Donut chart: sales mix by category (iPhone, Mac, iPad, Watch, AirPods, Vision Pro, TV & Home, Accessories)
- Best-sellers table: top 10 SKUs by units and revenue
- Live activity feed (new order, payment received, low stock, trade-in request, review posted)
- Quick actions: New Order, Add Product, Goods Received, Trade-In Intake, Reports
- Data export: orders, revenue, and inventory as CSV/Excel

3.8 Notifications & Alerts

- Email confirmation to customer on order creation and every status change
- SMS alert when order is Ready for Pickup or Out for Delivery (configurable)
- Back-in-stock email for wishlist items
- Pre-order status updates (stock arrived, payment due)
- In-app notification badge on nav bar for unread events
- Admin alert when SKU stock falls below its threshold

4. Functional Requirements

Authentication & Authorization

REQ ID	Description	Priority
FR-AUTH-01	The system shall allow users to register with email, password, full name, phone, and NIC.	High
FR-AUTH-02	The system shall authenticate users using JWT tokens with 15-minute expiry and refresh-token rotation.	High
FR-AUTH-03	The system shall enforce role-based access: CUSTOMER, STAFF, ADMIN.	High
FR-AUTH-04	The system shall allow password reset via a time-limited email link (expires in 1 hour).	High
FR-AUTH-05	The system shall log all login attempts (success/failure) with IP and timestamp.	Medium
FR-AUTH-06	The system shall support Google OAuth 2.0 as an alternative login method.	Low

Product Catalogue & Variants

REQ ID	Description	Priority
FR-PROD-01	Admin shall be able to create a product with: name, category, model number, description, release year, category-specific specs, and images.	High
FR-PROD-02	Admin shall be able to define variants per product as colour × storage/RAM combinations, each with a unique SKU, price (LKR), and stock quantity.	High
FR-PROD-03	The product detail page shall update price, images, and stock availability dynamically when the customer changes the selected colour or storage.	High
FR-PROD-04	The system shall support category-specific specification fields stored as flexible sub-documents (e.g., chip, camera for iPhone; RAM, display for Mac).	High
FR-PROD-05	The system shall support product condition types: Brand New, Refurbished, and Pre-Owned with grade (A/B/C).	Medium
FR-PROD-06	Customers shall be able to compare up to 3 products of the same category side by side.	Medium
FR-PROD-07	The system shall provide full-text search with autocomplete across product name, model number, and chip using MongoDB text/Atlas Search indexes.	Medium

Cart & Checkout

REQ ID	Description	Priority
FR-CART-01	The system shall maintain a persistent server-side cart per customer, merged with the guest cart on login.	High
FR-CART-02	The system shall re-validate stock for every cart line at checkout and block confirmation if any SKU is insufficient.	High
FR-CART-03	The system shall calculate order total as: $\Sigma(\text{unit_price} \times \text{qty}) - \text{promo_discount} + \text{delivery_fee}$. Delivery fee is determined by district and waived above a configurable subtotal threshold (e.g., LKR 50,000). Prices are VAT-inclusive.	High
FR-CART-04	The system shall support promo codes with type (percentage/fixed), validity window, minimum subtotal, and per-customer usage limit.	Medium
FR-CART-05	Customers shall be able to choose delivery (address + district) or store pickup (branch selection) at checkout.	High

Order Management

REQ ID	Description	Priority
FR-ORD-01	Customers and Staff shall be able to place an order specifying: cart items (SKUs), fulfilment method, delivery address or pickup branch, and payment method.	High
FR-ORD-02	The system shall generate a unique order reference in the format #IS-YYYY-XXXXX.	High

REQ ID	Description	Priority
FR-ORD-03	Staff/Admin shall be able to progress order status: Pending → Confirmed → Processing → Shipped / Ready for Pickup → Delivered / Collected → Completed.	High
FR-ORD-04	Customers shall be able to cancel their own order while it is in Pending or Confirmed status; later cancellation requires Staff approval.	High
FR-ORD-05	The system shall atomically decrement SKU stock when an order is confirmed and restore it if the order is cancelled before fulfilment.	High
FR-ORD-06	Staff shall record the serial number (and IMEI for iPhones/iPads) of each fulfilled unit, linking it to the order line for warranty purposes.	High
FR-ORD-07	The system shall generate a PDF invoice for every confirmed order, downloadable by the customer.	High
FR-ORD-08	The system shall support a return/refund request workflow with reason, photo evidence, Staff approval, and stock re-intake.	Medium

Inventory & Stock Control

REQ ID	Description	Priority
FR-INV-01	The system shall track stock quantity at SKU level and display availability on the storefront in real time.	High
FR-INV-02	The system shall record every stock movement (purchase-in, sale, return, adjustment, trade-in intake) in an immutable ledger with user and timestamp.	High
FR-INV-03	The system shall alert the Admin (dashboard + email) when a SKU's stock falls below its configured threshold.	Medium
FR-INV-04	Admin shall be able to record goods-received entries with supplier, quantity, unit cost, and batch reference.	High
FR-INV-05	The system shall provide a stock valuation report (quantity × unit cost) grouped by category.	Medium

Trade-In & Pre-Orders

REQ ID	Description	Priority
FR-TRD-01	Customers shall be able to submit a trade-in request with device model, storage, condition questionnaire answers, and up to 5 photos.	Medium
FR-TRD-02	Staff shall grade the device (A/B/C/Reject) and issue a quote; the customer shall accept or decline within a configurable validity period.	Medium
FR-TRD-03	An accepted trade-in value shall be applicable as credit toward a new purchase in the same account.	Medium
FR-TRD-04	Admin shall be able to create pre-order campaigns with deposit amount and expected date; pre-orders shall convert to orders in queue sequence when stock arrives.	Medium

Customer Management

REQ ID	Description	Priority
FR-CUST-01	Admin shall be able to view, search, and filter the customer list by name, email, or date joined.	High
FR-CUST-02	The customer profile shall store: full name, NIC, email, phone, and multiple delivery addresses.	High
FR-CUST-03	Customers shall be able to post a review (rating 1–5 + comment) only for products they have purchased (verified-purchase badge).	Medium
FR-CUST-04	Admin shall be able to suspend a customer account, blocking checkout while preserving order history.	High

Payments & Invoicing

REQ ID	Description	Priority
FR-PAY-01	The system shall record payment transactions with method, amount, date, status, and reference.	High
FR-PAY-02	For Bank Transfer, customers shall upload a payment slip; Staff shall verify and mark the payment as confirmed.	High
FR-PAY-03	Staff shall be able to mark a payment as paid, partially paid, or refunded, with a mandatory note.	High
FR-PAY-04	The system shall compute refund amounts for approved returns based on a configurable policy (e.g., 100% within 7 days if unopened).	Medium
FR-PAY-05	The system shall integrate with PayHere for card payments and Koko/Mintpay for instalments (Phase 2).	Low

Reporting & Analytics

REQ ID	Description	Priority
FR-REP-01	Admin shall see a dashboard with KPIs: total revenue (monthly/annual), orders today, low-stock SKUs, and new customers.	High
FR-REP-02	The system shall provide a monthly revenue bar chart for the current year using MongoDB aggregation pipelines.	High
FR-REP-03	The system shall provide a sales-mix donut chart by product category.	Medium
FR-REP-04	Admin shall be able to export order, revenue, and inventory reports as CSV or Excel.	Medium
FR-REP-05	The system shall maintain a live activity feed showing the last 20 system events.	Medium

5. Non-Functional Requirements

REQ ID	Description	Priority
NFR-PERF-01	API endpoints shall respond in < 500 ms under 200 concurrent users (p95).	High
NFR-PERF-02	Catalogue and product-detail reads shall be served with appropriate MongoDB indexes; no unindexed (COLLSCAN) queries on hot paths.	High
NFR-PERF-03	The Angular SPA shall achieve a Lighthouse performance score of ≥ 85 on desktop.	Medium
NFR-SEC-01	All API communication shall use HTTPS (TLS 1.2+). HTTP requests shall redirect to HTTPS.	High
NFR-SEC-02	Passwords shall be hashed using bcrypt with a minimum cost factor of 12.	High
NFR-SEC-03	The API shall implement rate limiting: max 100 requests/minute per IP.	High
NFR-SEC-04	All user-supplied query values shall be passed as parameters via Spring Data MongoDB; string concatenation into queries and the \$where operator are prohibited (NoSQL-injection prevention).	High
NFR-SEC-05	Input validation shall be enforced on both Angular (reactive forms) and Spring Boot (Jakarta Bean Validation).	High
NFR-AVAIL-01	The system shall target 99.5% uptime (excluding planned maintenance windows).	High
NFR-AVAIL-02	MongoDB shall run as a 3-node replica set with automatic failover; daily backups (mongodump / Atlas snapshots) retained for 30 days.	High
NFR-SCALE-01	The backend shall be stateless and horizontally scalable behind a load balancer.	Medium
NFR-SCALE-02	The data model shall support 5,000+ products, 50,000+ SKUs, and 1,000,000+ orders without redesign; orders use referencing (not unbounded embedding) to respect the 16 MB document limit.	Medium
NFR-CONS-01	Stock decrement and order creation shall be executed in a MongoDB multi-document transaction (or an atomic findAndModify pattern) to guarantee no overselling.	High
NFR-MAINT-01	Backend code shall achieve $\geq 80\%$ unit-test coverage (JUnit 5 / Mockito / Testcontainers with MongoDB module).	Medium
NFR-MAINT-02	API shall follow OpenAPI 3 specification; Swagger UI shall be available in dev profile.	Medium
NFR-ACCESS-01	The Angular UI shall meet WCAG 2.1 AA accessibility standards.	Low
NFR-I18N-01	UI text strings shall be externalised to support future localisation (Sinhala/Tamil).	Low

6. External Interface Requirements

6.1 User Interfaces

The Angular 17 SPA is the sole UI. It communicates with the API via HttpClient using JSON. Angular Material or a custom Apple-inspired component library is used for form controls, tables, and modals. The UI is responsive (mobile-first, breakpoints at 768 px and 1280 px) with a clean, minimal aesthetic consistent with the Apple brand feel.

6.2 Backend REST API

Spring Boot exposes RESTful endpoints under the base path /api/v1/. All requests and responses use application/json. Authentication is via Authorization: Bearer {JWT} header. Pagination uses page/size query parameters. Error responses follow RFC 7807 Problem Details.

6.3 Email Service

Spring Mail with SendGrid SMTP sends transactional emails (order confirmation, status updates, back-in-stock, password reset). HTML email templates (Thymeleaf) are stored in resources/templates.

6.4 SMS Gateway (Optional)

Dialog SMS API (or Twilio) is used for order pickup/delivery alerts. Configured via environment variables; disabled if credentials are absent.

6.5 File Storage

AWS S3 (or MinIO for on-prem) stores product images, payment slips, and trade-in photos. The API returns pre-signed URLs. Max file size: 5 MB per upload.

6.6 Payment Gateway (Phase 2)

PayHere hosted checkout for card payments and Koko/Mintpay for instalment plans. The API verifies gateway callbacks via HMAC signature before marking payments as confirmed.

7. Database Design Overview (MongoDB)

The MongoDB 7 database (name: **istore_db**) uses a document model designed around access patterns. The guiding rule is: **embed what is read together, reference what grows unboundedly**. Variants are embedded inside products (bounded, always read with the product); orders reference products by SKU and snapshot the price/name at purchase time so historical orders are immune to later catalogue changes. Full schema validation rules (\$jsonSchema) are provided as a separate deliverable (schema-validation.js).

7.1 Collections

users	All system users. role field: CUSTOMER, STAFF, ADMIN. Embeds addresses[] for customers.
products	Catalogue master. Embeds specs (flexible per category) and variants[] (SKU, colour, storage, price, stock, condition).
categories	Lookup: iPhone, Mac, iPad, Watch, AirPods, Vision Pro, TV & Home, Accessories — with the spec template per category.
carts	One active cart per customer. Embeds items[] {sku, productId, qty, unitPrice}.
orders	Core transaction. Embeds items[] (SKU + snapshotted name/price), delivery info, payment summary, status history[].
payments	Payment transactions: orderId (ref), method, amount, status, slipUrl, gateway reference.
stock_movements	Immutable ledger: sku, type (IN/SALE/RETURN/ADJUST/TRADE_IN), qty, unitCost, userId, timestamp.
trade_ins	Trade-in requests: device details, photos[], grade, quoteAmount, status, linked orderId.
pre_orders	Pre-order queue: productId, variant sku, customerId, deposit, queue position, status.
warranties	Registered devices: serialNumber, imei, orderId (ref), customerId, purchaseDate, expiryDate, claims[].
reviews	Product reviews: productId (ref), customerId, rating, comment, verifiedPurchase flag.
promo_codes	Discount codes: type, value, validFrom/To, minSubtotal, usageLimit, usedBy[].
notifications	In-app notification log per user (type, message, read flag).
audit_logs	Immutable event log for ADMIN (entity, action, userId, before/after documents).

7.2 Sample Product Document

```
{
  "_id": ObjectId("..."),
  "name": "iPhone 16 Pro",
  "slug": "iphone-16-pro",
  "category": "iPhone",
  "modelNumber": "A3101",
  "releaseYear": 2024,
  "status": "ACTIVE",
  "specs": {                                // flexible per category
    "chip": "A18 Pro",
    "display": "6.3-inch Super Retina XDR",
    "camera": "48MP Fusion | 48MP Ultra Wide | 12MP Telephoto",
    "battery": "Up to 27 hours video playback",
    "connectivity": ["5G", "Wi-Fi 7", "USB-C"]
  },
  "variants": [
    {
      "sku": "IP16P-256-NAT",
      "color": { "name": "Natural Titanium", "hex": "#c2bcb2" },
      "storage": "256GB",
      "condition": "BRAND_NEW",
      "price": NumberDecimal("434900.00"), // LKR
      "stock": 12,
      "lowStockThreshold": 3,
      "images": ["https://cdn.istore.lk/ip16p/nat-1.jpg"]
    }
  ]
}
```

```

    },
    {
      "sku": "IP16P-512-BLU",
      "color": { "name": "Blue Titanium", "hex": "#3f4a5a" },
      "storage": "512GB",
      "condition": "BRAND_NEW",
      "price": NumberDecimal("512900.00"),
      "stock": 4,
      "lowStockThreshold": 3,
      "images": ["https://cdn.istore.lk/ip16p/blu-1.jpg"]
    }
  ],
  "rating": { "avg": 4.8, "count": 57 },
  "createdAt": ISODate("2026-07-01T08:30:00Z"),
  "updatedAt": ISODate("2026-07-10T14:02:00Z")
}

```

7.3 Key Relationships & Design Decisions

- products **embeds** variants[] — bounded set (typically < 20), always read together with the product.
- orders **references** products via SKU but **snapshots** name and price at purchase time (immutable history).
- orders **embeds** statusHistory[] — small, bounded, always displayed with the order.
- reviews and stock_movements are **separate collections** (unbounded growth would breach the 16 MB document limit if embedded).
- Stock decrement uses an atomic conditional update — updateOne({ "variants.sku": sku, "variants.stock": { \$gte: qty } }, { \$inc: { "variants.\$stock": -qty } }) — inside a multi-document transaction with order creation.
- Monetary values use Decimal128 (NumberDecimal), never floating point.

7.4 Indexes

products	{ slug: 1 } unique · { category: 1, status: 1 } · { "variants.sku": 1 } unique · text index on name/modelNumber
orders	{ orderNo: 1 } unique · { customerId: 1, createdAt: -1 } · { status: 1, createdAt: -1 }
users	{ email: 1 } unique · { nic: 1 } unique sparse
stock_movements	{ sku: 1, createdAt: -1 }
reviews	{ productId: 1, createdAt: -1 } · { customerId: 1, productId: 1 } unique
warranties	{ serialNumber: 1 } unique · { customerId: 1 }

8. Technology Stack

Frontend	Angular v17 — standalone components, signals
UI Library	Angular Material / custom Apple-inspired components, WCAG 2.1 AA
HTTP Client	Angular HttpClient — interceptors for JWT injection & errors
State Management	NgRx / RxJS — store for auth, cart, and catalogue state
Backend	Spring Boot v3.x on JDK 21
Security	Spring Security + JWT (io.jsonwebtoken / jjwt)
Data Access	Spring Data MongoDB — repositories + MongoTemplate for aggregations
Transactions	MongoDB multi-document transactions (replica set required)
Validation	Jakarta Bean Validation (Hibernate Validator)
API Docs	SpringDoc OpenAPI — Swagger UI on /swagger-ui.html
Database	MongoDB v7.0 — 3-node replica set / Atlas; Decimal128 for money
Migrations	Mongock — version-controlled schema/seed migrations for MongoDB
Search	MongoDB text indexes (V1) → Atlas Search (Phase 2)
Email	Spring Mail + SendGrid — HTML Thymeleaf templates
File Storage	AWS S3 / MinIO — pre-signed URLs
Containerisation	Docker + Docker Compose (app + mongo replica set) — dev & prod parity
Testing (BE)	JUnit 5, Mockito, Testcontainers (MongoDB module) — ≥ 80% coverage target
Testing (FE)	Jasmine, Karma, Cypress — E2E critical flows (checkout, order)
CI/CD	GitHub Actions — Build → Test → Docker Push → Deploy

9. Constraints & Risks

9.1 Constraints

- The system must operate on MongoDB 7; migration to a relational database is out of scope.
- Multi-document transactions require a replica set — standalone mongod is not supported in production.
- The Angular SPA must work without a native mobile app in V1; PWA capabilities are optional.
- Payment gateway integration (PayHere, Koko/Mintpay) is deferred to Phase 2 due to merchant onboarding time; V1 supports Cash on Delivery and verified Bank Transfer only.
- All UI text must support Unicode to accommodate future Sinhala/Tamil localisation.
- PDPA compliance: customer PII must be deletable/anonymisable on request; order and audit records are anonymised (not deleted) to preserve financial history.

9.2 Risks

Risk	Likelihood	Impact	Mitigation
Overselling stock under concurrent checkouts	Medium	High	Atomic conditional \$inc update + multi-document transaction; reserved-stock TTL during checkout
Schema drift due to MongoDB's flexible schema	Medium	Medium	\$jsonSchema validation on collections; Mongock migrations; DTO validation in Spring
Unbounded array growth breaching 16 MB doc limit	Low	High	Reviews & stock movements kept in separate collections; embed only bounded data
Grey-market / counterfeit product disputes	Low	High	Serial/IMEI capture at fulfilment; warranty registration tied to distributor records
Third-party SMS/email downtime	Low	Medium	Async retry queue; fallback to email-only
JWT secret compromise	Low	High	Secret rotation procedure; short token expiry; refresh-token revocation list
Scope creep from stakeholders	High	Medium	Strict change-request process; MoSCoW prioritisation

10. Appendix

10.1 Angular Module Structure (Recommended)

CoreModule	Singleton services: AuthService, ApiService, JwtInterceptor, ErrorInterceptor
SharedModule	Reusable components: LoaderComponent, BadgeComponent, ConfirmDialogComponent, VariantPickerComponent
AuthModule	Login, Register, ForgotPassword, ResetPassword pages
StoreModule	Home/Hero, ProductCatalogue, ProductDetail (variant selector), Compare, Wishlist
CheckoutModule	Cart, CheckoutStepper (delivery → payment → confirm), OrderConfirmation
AccountModule	MyOrders, OrderDetail, TradeInRequests, Warranties, Profile
AdminModule	Dashboard, ProductAdmin, InventoryAdmin, OrderAdmin, TradeInAdmin, CustomerAdmin, Reports (lazy-loaded)

10.2 Spring Boot Package Structure (Recommended)

com.istore.api.config	SecurityConfig, JwtConfig, MongoConfig (transactions), SwaggerConfig, CorsConfig
com.istore.api.auth	AuthController, AuthService, JwtService, RefreshTokenService
com.istore.api.user	UserController, UserService, UserRepository, User (document)
com.istore.api.product	ProductController, ProductService, ProductRepository, Product, Variant
com.istore.api.cart	CartController, CartService, Cart
com.istore.api.order	OrderController, OrderService, OrderRepository, Order (with @Transactional stock logic)
com.istore.api.inventory	InventoryController, StockMovementService, StockMovement
com.istore.api.tradein	TradeInController, TradeInService, TradeIn
com.istore.api.payment	PaymentController, PaymentService, Payment, InvoiceService (PDF)
com.istore.api.notification	NotificationService, EmailService, SmsService
com.istore.api.report	ReportController, ReportService (MongoDB aggregation pipelines)
com.istore.api.exception	GlobalExceptionHandler, AppException, ErrorResponse

10.3 Key API Endpoints Summary

Method	Endpoint	Role	Description
POST	/api/v1/auth/register	Public	Register new user
POST	/api/v1/auth/login	Public	Authenticate, returns JWT
POST	/api/v1/auth/refresh	Public	Refresh access token
GET	/api/v1/products	Public	List products (paginated, filterable by category)
GET	/api/v1/products/{slug}	Public	Product detail with variants
POST	/api/v1/products	ADMIN	Create product with variants
PATCH	/api/v1/products/{id}/variants/{sku}	ADMIN	Update variant price/stock
GET	/api/v1/cart	CUSTOMER	Get current cart
POST	/api/v1/cart/items	CUSTOMER	Add SKU to cart
POST	/api/v1/orders	CUSTOMER+	Place order (transactional stock decrement)

Method	Endpoint	Role	Description
GET	/api/v1/orders	STAFF+	List orders (paginated)
PATCH	/api/v1/orders/{id}/status	STAFF+	Update order status
POST	/api/v1/orders/{id}/fulfil	STAFF	Record serial/IMEI at fulfilment
POST	/api/v1/inventory/goods-received	ADMIN	Record stock-in
POST	/api/v1/trade-ins	CUSTOMER	Submit trade-in request
PATCH	/api/v1/trade-ins/{id}/quote	STAFF	Grade device & issue quote
GET	/api/v1/reports/dashboard	ADMIN	Dashboard KPIs (aggregation)
GET	/api/v1/reports/export	ADMIN	Export CSV/Excel

This SRS is a living document. Changes must go through the formal Change Request process. Current version: 1.1 — July 2026